A APPENDIX: END-TO-END MISCOMPILATION BY BOTH LLVM AND GCC

This C program is miscompiled by both GCC and LLVM:[7]

```
$ cat b.c
void f(int *x, int *y) {}

$ cat a.c
#include <stdint.h>
#include <stdio.h>

void f(int *, int *);

int main(void) {
  int a = 0, y[1], x = 0;
  uintptr_t pi = (uintptr_t)&x;
  uintptr_t yi = (uintptr_t)(y + 1);
  int n = pi != yi;

  if (n) {
    a = 100;
    pi = yi;
  }

  if (n) {
    a = 100;
    pi = (uintptr_t)y;
  }

  *(int *)pi = 15;

  printf("a=%d x=%d\n", a, x);

  f(&x, y);

  return 0;
}

$ clang-5.0 -Wall -O2 a.c b.c ; ./a.out
a=0 x=0
$ gcc-7 -Wall -O2 a.c b.c ; ./a.out
a=0 x=0
```

The result produced by both compilers is incorrect. There are only two possible outcomes, depending on whether `x` and `y` are allocated consecutively (`n` is false) or not (`n` is true):

- Case 1: `n` is true and the program must print
  `a=100 x=0`
- Case 2: `n` is false and the program must print
  `a=0 x=15`

[7]Please note that the program may produce different results on different platforms, since the miscompilation is only observable with some stack layouts. Changing the order of declaration of variables `x` and `y` is often sufficient to observe the miscompilation.